Natural Language Processing

Basic Text Processing

Regular Expressions

Lec. 2

Regular Expressions

Python for Everybody

Regular Expressions

- In computing, a regular expression, also referred to as "regex" or "regexp", provides a concise and flexible means for matching strings of text, such as particular characters, words, or patterns of characters.
- A regular expression is written in a formal language that can be interpreted by a regular expression processor.

Regular Expressions

Really clever “wild card” expressions for matching and parsing strings

Regular expression – Wikipedia, the free encyclopedia

http://en.wikipedia.org/wiki/Regular_expression

More than 100 matches | regular | Done

Log in / create account

Article | Discussion | Read | Edit | View history | Search

WIKIPEDIA
The Free Encyclopedia

Main page
Contents
Featured content
Current events
Random article
Donate to Wikipedia

Interaction
Help
About Wikipedia
Community portal
Recent changes
Contact Wikipedia

Regular expression

From Wikipedia, the free encyclopedia

In computing, a **regular expression**, also referred to as **regex** or **regexp**, provides a concise and flexible means for matching strings of text, such as particular characters, words, or patterns of characters. A regular expression is written in a formal language that can be interpreted by a regular expression processor, a program that either serves as a parser generator or examines text and identifies parts that match the provided specification.

The following examples illustrate a few specifications that could be expressed in a regular expression:

- The sequence of characters "car" appearing consecutively in any context, such as in "car", "cartoon", or "bicarbonate"
- The sequence of characters "car" occurring in that order with other characters between them, such as in "Icelander" or "chandler"

Really smart “Find” or “Search”

Understanding Regular Expressions

Very powerful and quite cryptic

Fun once you understand them

Regular expressions are a language unto themselves

A language of “marker characters” - programming with characters

It is kind of an “old school” language - compact

WHENEVER I LEARN A NEW SKILL I CONCOCT ELABORATE FANTASY SCENARIOS WHERE IT LETS ME SAVE THE DAY.

OH NO! THE KILLER MUST HAVE FOLLOWED HER ON VACATION!

BUT TO FIND THEM WE'D HAVE TO SEARCH THROUGH 200 MB OF EMAILS LOOKING FOR SOMETHING FORMATTED LIKE AN ADDRESS!

IT'S HOPELESS!

EVERYBODY STAND BACK.

I KNOW REGULAR EXPRESSIONS.

PERL!

TAP TAP

Regular Expression Quick Guide

```
^         Matches the beginning of a line
$         Matches the end of the line
.         Matches any character
\s        Matches whitespace
\S        Matches any non-whitespace character
*         Repeats a character zero or more times
*?        Repeats a character zero or more times (non-greedy)
+         Repeats a character one or more times
+?        Repeats a character one or more times (non-greedy)
[aeiou]   Matches a single character in the listed set
[^XYZ]    Matches a single character not in the listed set
[a-z0-9]  The set of characters can include a range
(         Indicates where string extraction is to start
)         Indicates where string extraction is to end
```

Regular Expressions: Disjunctions

How can we search for any of these?

- woodchuck
- woodchucks
- Woodchuck
- Woodchucks

Regular Expressions: Disjunctions

Letters inside square brackets []

Pattern	Matches
`[wW]oodchuck`	Woodchuck, woodchuck
`[1234567890]`	Any digit

Ranges `[A-Z]`

Pattern	Matches	
`[A-Z]`	An upper case letter	`Drenched Blossoms`
`[a-z]`	A lower case letter	`my beans were impatient`
`[0-9]`	A single digit	`Chapter 1: Down the Rabbit Hole`

Regular Expressions: Negation in Disjunction

Negations `[^Ss]`

- Carat means negation only when first in []

Pattern	Matches	
`[^A-Z]`	Not an upper case letter	`Oyfn pripetchik`
`[^Ss]`	Neither ‘S’ nor ‘s’	`I have no exquisite reason”`

Regular Expressions: More Disjunction

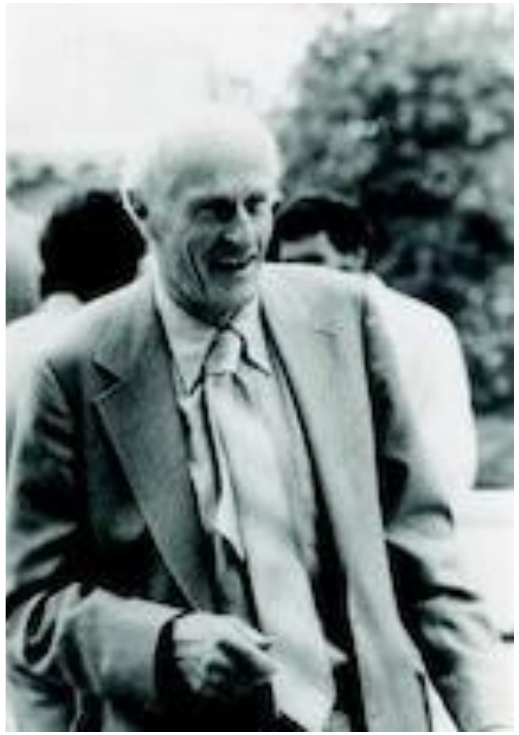
Woodchuck is another name for groundhog!

The pipe | for disjunction

Pattern	Matches
`groundhog|woodchuck`	`woodchuck`
`yours|mine`	`yours`
`a|b|c`	= [abc]
`[gG]roundhog|[Ww]oodchuck`	`Woodchuck`

Regular Expressions: ? *+ .

Pattern	Matches	
colou?r	Optional previous char	color colour
oo*h!	0 or more of previous char	oh! ooh! oooh! ooooh!
o+h!	1 or more of previous char	oh! ooh! oooh! ooooh!
baa+		baa baaa baaaa baaaaa
beg.n		begin begun begun beg3n



Stephen C Kleene

Kleene *, Kleene +

Regular Expressions: Anchors ^ $

Pattern	Matches
^[A-Z]	Palo Alto
^[^A-Za-z]	1 "Hello"
\.$	The end.
.$	The end? The end!

The Regular Expression Module

- **Before you can use regular expressions in your program, you must import the library using “import re”**
- **You can use re.search() to see if a string matches a regular expression, similar to using the find() method for strings**
- **You can use re.findall() to extract portions of a string that match your regular expression, similar to a combination of find() and slicing: var[5:10]**

Using re.search() Like find()

```
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if line.find('From:') >= 0:
        print(line)
```

```
import re

hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('From:', line) :
        print(line)
```

Welcome To Colaboratory - Co | Untitled9.ipynb - Colaboratory

https://colab.research.google.com/drive/1rK

+ Code + Text

Reguluar Expression

```
[1] import re
```

```
[ ] #re has main function (search and findall)
```

Search Function: return the first match object

```
[2] re.search('2',"hello level 2")
```

```
<re.Match object; span=(12, 13), match='2'>
```

localhost:8888/lab

File Edit View Run Kernel Tabs Settings Help

Launcher | inheritance.ipynb | regexp.ipynb

Code | Python 3

[1]:
```
s = 'hello world :)'
s.find('hel')
```

[1]: 0

[3]:
```
s.find('w')
```

[3]: 6

Using re.search() Like startswith()

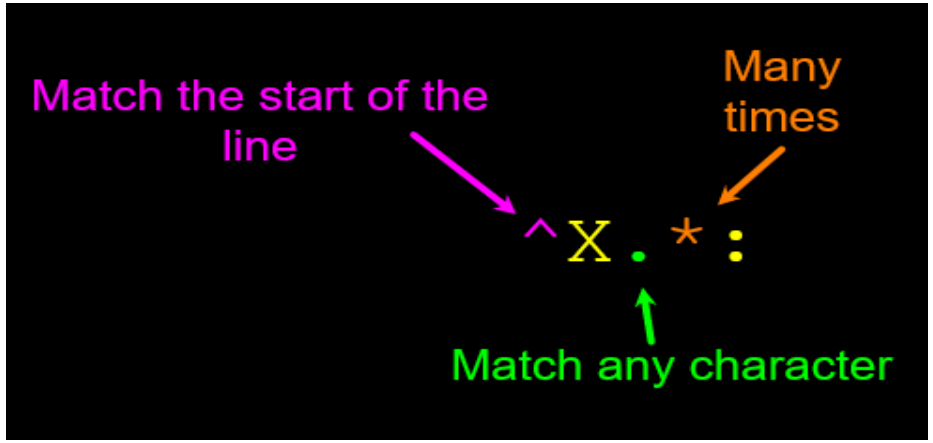
```
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if line.startswith('From:') :
        print(line)
```

```
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('^From:', line) :
        print(line)
```

Wild-Card Characters

- The dot character matches any character
- If you add the asterisk character, the character is “any number of times”

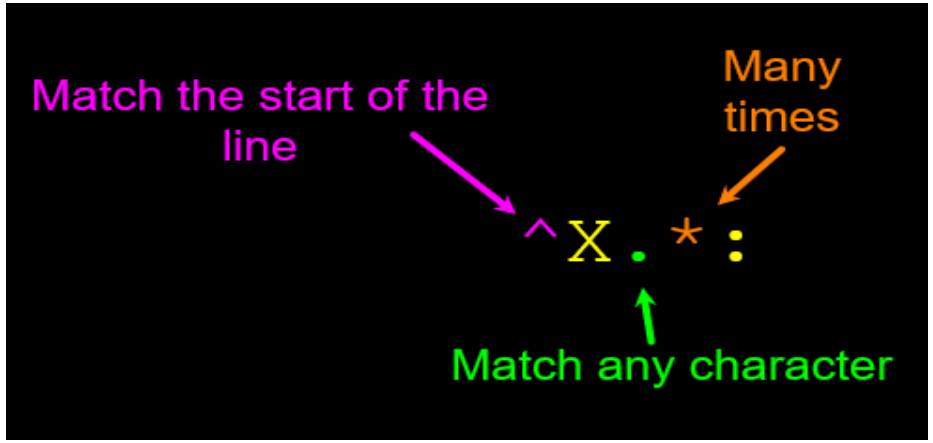
```
X-Sieve: CMU Sieve 2.3
X-DSPAM-Result: Innocent
X-DSPAM-Confidence: 0.8475
X-Content-Type-Message-Body:
```



Fine-Tuning Your Match

- Depending on how "clean" your data is and the purpose of your application, you may want to narrow your match down a bit

```
X-Sieve: CMU Sieve 2.3
X-DSPAM-Result: Innocent
X-Plane is behind schedule: two
X-   : Very Short
```



Fine-Tuning Your Match

Depending on how “clean” your data is and the purpose of your application, you may want to narrow your match down a bit

```
X-Sieve: CMU Sieve 2.3
X-DSPAM-Result: Innocent
X-: Very Short
X-Plane is behind schedule: two weeks
```

Match the start of the line

One or more times

```
^X-\S+:
```

Match any non-whitespace character

Matching and Extracting Data

- re.search() returns a True/False depending on whether the string matches the regular expression
- If we actually want the matching strings to be extracted, we use re.findall()

[0-9]+

One or more digits

```
>>> import re
>>> x = 'My 2 favorite numbers are 19 and 42'
>>> y = re.findall('[0-9]+',x)
>>> print(y)
['2', '19', '42']
```

Matching and Extracting Data

When we use re.findall(), it returns a list of zero or more sub-strings that match the regular expression

```
>>> import re
>>> x = 'My 2 favorite numbers are 19 and 42'
>>> y = re.findall('[0-9]+',x)
>>> print(y)
['2', '19', '42']
>>> y = re.findall('[AEIOU]+',x)
>>> print(y)
[]
```

Warning: Greedy Matching

The repeat characters (* and +) push outward in both directions (greedy) to match the largest possible string

```
>>> import re
>>> x = 'From: Using the : character'
>>> y = re.findall('^F.+:', x)
>>> print(y)
['From: Using the :']
```

Why not 'From:' ?

One or more characters

^F.+:

First character in the match is an F

Last character in the match is a :

Non-Greedy Matching

Not all regular expression repeat codes are greedy!
If you add a ? character, the + and * chill out a bit...

```
>>> import re
>>> x = 'From: Using the : character'
>>> y = re.findall('^F.+?:', x)
>>> print(y)
['From:']
```

One or more characters but not greedy

^F.+?:

First character in the match is an F

Last character in the match is a :

Fine-Tuning String Extraction

You can refine the match for re.findall() and separately determine which portion of the match is to be extracted by using parentheses

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
```

```
>>> y = re.findall('\S+@\S+',x)
>>> print(y)
['stephen.marquard@uct.ac.za']
```

\S+@\S+

At least one non-whitespace character

Fine-Tuning String Extraction

Parentheses are not part of the match - but they tell where to start and stop what string to extract

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
```

```
>>> y = re.findall('\S+@\S+',x)
>>> print(y)
['stephen.marquard@uct.ac.za']
>>> y = re.findall('^From (\S+@\S+)',x)
>>> print(y)
['stephen.marquard@uct.ac.za']
```

```
^From (\S+@\S+)
```

QUIZ TIME

1- Use the Python programming language to create pictures out of text.

main.py

```python
print('Hi, I can code in Python!')

print('here\'s a picture of a dog:')
print('o___ ')
print(' |||| ")
```

NOTE: **The dog ’ s legs are made using the pipe character | which you can type by pressing Shift + \ on most UK/US English keyboards.**

2- Did you know that you can also calculate text?!

What will the following program print to the screen? See if you can guess it correctly before running the program.

```
print('ha ' * 4)
 print('ba' + 'na' *2)
print('Hello' + '!' *10)
```

Any Question?